VRIJE
UNIVERSITEIT
BRUSSEL

Graduation thesis submitted in partial fulfilment of the requirements for the degree of de Ingenieurswetenschappen: Computerwetenschappen

PROGRAMMING ROBOTS WITH HIGHER-ORDER REACTIVE PROGRAMMING

Gérard Lichtert

2025-2026

Promotor: Prof. Dr. Wolfgang De Meuter
Advisor: Bjarno Oeyen

Sciences and Bio-engineering Sciences

VRIJE
UNIVERSITEIT
BRUSSEL

Proefschrift ingediend met het oog op het behalen van de graad van Master of Science in de Ingenieurswetenschappen: Computerwetenschappen

ROBOTS PROGRAMMEREN MET HOGERE-ORDE REACTIEF PROGRAMMEREN

Gérard Lichtert

2025-2026

Promotor: Prof. Dr. Wolfgang De Meuter
Advisor: Bjarno Oeyen

Wetenschappen en Bio-ingenieurswetenschappen

Abstract

Reactive programming (RP) is a declarative programming paradigm based on data streams and the propagation of change. In a nutshell, if we have two variables x and y and another variable z that sums both x and y, it should be sufficient to update either x or y and z should be updated automatically.

There are several approaches to achieve reactive programming. One approach is to create a dedicated language. In this case the reactivity will be native to the language. Another option is to extend a language with reactive features, like

Contents

Chapter 1

Introduction

Modern software is often written in such a way that it is event-driven, or in other words, reactive. This means that applications reacts to an incoming event by creating subsequent events that arise as a reaction to the incoming event. Such an application is often called a reactive system. Reactive systems are often implemented with callbacks or by implementing the Observer Pattern. However, this does not come without its drawbacks as this may lead to 'inversion of control' or 'callback hell'[1][2]. To alleviate this we introduce **Reactive Programming**. A programming paradigm that allows the user to write reactive systems in a declarative way. Traditionally the sum and the product of two elements would be written as shown in Listing 1.

```
int x = 5;
int y = 10;
int s = x + y;
int p = x * y;
// x changes state
x = 10;
// s requires recomputation
s = x + y;
// p requires recomputation
p = x * y;

```

Listing 1: Imperative product and sum

Whenever x or y would change state, you would have to manually update s and p. Reactive programming allows you to declare x and y as sources of data and s and p as the result of the relevant computations. Whenever the value of x or y would change, the variables dependent on the sources would be automatically recomputed, and the result propagated further in the reactive system. Schematically the dependency graph of product-sum would look like Figure 1.1

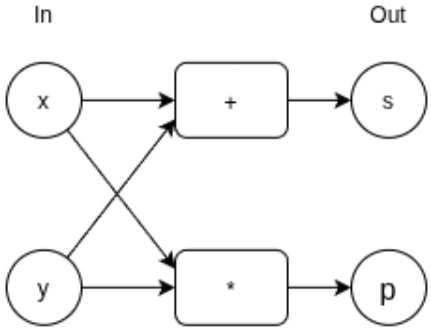

Figure 1.1: Dependency graph for product-sum

There are different ways of how such a reactive system could be created using reactive programming. One way is to use a programming which itself is natively reactive. A few examples are Haai [3], Another is to add native reactivity to an existing language by extending it the language or by implementing a reactive library such as ReactiveX. We will be looking at the former and more specifically the higher-order reactive programming language **Haai**, which was developed at the SOFT lab at the VUB. Haai uses reactors as their As a preview, an example of how such a product and sum could be written in Haai is shown in Listing 2. Haai uses reactors as its native way of declaring a reactive program.

```
(defr (product-sum x y)
    (def sum (+ x y))
    (def product (* x y))
    (out product sum))

```

Listing 2: Product sum in Haai

Additionally, we want to delve into the world of electronics. From maintaining your fridge's temperature to controlling a drones' altitude, electronics are deeply embedded in everyone's day-to-day life. Writing software for these electronics can also be challenging. This mainly due to the fact that using peripherals require using their associated libraries, which are not always standardised. Another challenge is that creating programs for microcontrollers often require the developer to use a low-level programming language such as C/C++ or Rust, which come with their own challenges. Nowadays, there are some efforts being made to provide a standardised collection of tools to aid with the development of systems with many peripherals, or robots. There is Robot Operating Systems (ROS) [4], Open Robot Control Software (OROCOS) [5], Yet Another Robot Platform (YARP) [6] to name a few. We will be focussing on using **(ROS)**.

1.1 Contributions

Reactive programming is used across a multitude of domains. It is used in web and mobile development for instance. However, we will be focussing on using it for the Internet of Things

(IoT). IoT consists of multiple physical objects, usually with very limited computing power and/or memory. In this thesis we introduce a way of declaring a reactive program across multiple devices from a single source. This means that the developer declares the program once, along with declarations where which part of the program lives and then deploys the program to the relevant devices making use of ROS. This is achieved by the following components:

- An extension of the Remus VM [7], more specifically, extending the language to accept a declaration of where which part of the program lives. The Remus compiler will use this metadata to split the Haai reactor declarations into an embedded part and a server part. Each piece will then be available for the user to be deployed in their respective parts of the IoT network.

- A new tool called CHaai, which translates the Remus bytecode obtained from compiling the Haai program into a C program. The C program will be required to run the program on microcontrollers using ROS.

- A C port of the existing Remus VM written in rust [8], also created to allow developers to create reactive programs for microcontrollers. However, it is adapted with respects to the current Instruction Set Architecture (ISA) of the Racket version of Remus [7] and allows usage of ROS.

1.2 Overview of this Dissertation

Chapter 2 gives the problem statement as well as an overview of the current reactive programming solutions and existing toolkits to program networks of microcontrollers. It also introduces a concept called 'Tierless programming', which will inspire the declarative way of telling Remus where which part of the reactive program lives.

Chapter 3 introduces μ-Remus. A C port of the Remus VM, which was previously written in Rust as well as in Racket. It explains why the C port was necessary as well as conceptualizes the link between the ROS communication protocols and the dependency graph generated by reactive programming.

Chapter 4 demonstrates usage of the Haai language extensions inspired by Tierless programming to declare where which part of the reactive program lives. It will also tell you about the CHaai tool. Which will create a C version of the reactive program. It is ported to C to be compatible with ROS.

Chapter 5 will evaluate the tools and extensions driven by an example. The target microcontroller will be a Raspberry Pi Pico H (2021) and will be used along a PC.

Lastly, chapter 6 will conclude this dissertation and give notes on some possible future work.

1.3 Approach

We first start by discussing the state of the art, what tools already exist and then move on to the problem statement and continue with the contributions part of the dissertation. However, since it is heavily dependent on a language and a toolkit we will first give a brief introduction into some relevant parts of the language or toolkit, each time providing the necessary context and information, prior to moving on to the actual contributions.

Chapter 2

State of the Art

2.1 Reactive Programming

2.2 RP languages targeting MCUs

2.3 Systems to program robots

2.4 Tierless Programming

2.5 Problem Statement

Chapter 3

μ-Remus

3.1 Haai & Remus

3.2 ROS & μ-ROS

3.3 Mapping RP to ROS & μ-ROS

3.4 μ-Remus: A C implementation

Chapter 4

Programming Robots with Reactors

4.1 Linking ROS and μ-ROS

Chapter 5

Tierless Haai

5.1 Haai to CHaai

Chapter 6

Evaluation

6.1 Evaluating Haai distribution

6.2 Evaluating Haai to CHaai

6.3 Evaluating CHaai on μ-Remus

Chapter 7

Conclusion & Future Work

Bibliography

[1] S. Van den Vonder, B. Oeyen, G. Salvaneschi, W. De Meuter and J. Nicolay, 'A survey on reactive programming and reactive streams,' *ACM Computing Surveys*, Nov. 2024, Manuscript submitted to ACM.

[2] E. Bainomugisha, A. Lombide Carreton, T. Van Cutsem, S. Mostinckx and W. De Meuter, 'A survey on reactive programming,' *ACM Comput. Surv.*, vol. 45, no. 4, Aug. 2013, ISSN: 0360-0300. DOI: 10.1145/2501654.2501666 [Online]. Available: https://doi.org/10.1145/2501654.2501666

[3] B. Oeyen, J. De Koster and W. De Meuter, 'Reactive programming without functions,' *The Art, Science, and Engineering of Programming*, vol. 8, no. 3, 11:1–11:30, 2024. DOI: 10.22152/programming-journal.org/2024/8/11

[4] A. A. Allaban, D. Bonnie, E. Knapp, P. Gokhale and T. Moulard, 'Developing production-grade applications with ros 2,' in *Robot Operating System (ROS): The Complete Reference (Volume 6)*, ser. Studies in Computational Intelligence, A. Koubaa, Ed., vol. 962, Cham, Switzerland: Springer Nature Switzerland AG, 2021, pp. 3–52, ISBN: 978-3-030-75471-6. DOI: 10.1007/978-3-030-75472-3_1

[5] H. Bruyninckx, 'Open robot control software: The OROCOS project,' in *Proceedings of the 2001 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 3, Seoul, South Korea: IEEE, 2001, pp. 2523–2528, ISBN: 0-7803-6475-9. DOI: 10.1109/ROBOT.2001.933002

[6] G. Metta, P. Fitzpatrick and L. Natale, 'YARP: Yet another robot platform,' *International Journal of Advanced Robotic Systems*, vol. 3, no. 1, pp. 43–48, 2006. DOI: 10.5772/5761

[7] B. Oeyen, J. Nicolay and W. De Meuter, 'A virtual machine for higher-order reactors,' in *Programming Companion 2024: Proceedings of the 8th International Conference on the Art, Science, and Engineering of Programming*, E. Soderberg and L. Church, Eds., New York, NY, USA: Association for Computing Machinery, 2024, pp. 52–56. DOI: 10.1145/3660829.3660840

[8] C. Descheemaeker, 'Compilation-based execution and optimisation of reactive programs on the bare metal: A vm approach,' Master's thesis, Vrije Universiteit Brussel, Brussels, Belgium, 2023.